

Array:-

D&A Java does not support pointers
∴ We cannot access elements of array using indices.

Stack:-

```
→ import java.util.Scanner;
public class StackArrayIntDemo {
    private final int[] stack;
    private int top = (-1);
    // capacity is unknown that is why we need to
    // make a constructor:-
    StackArrayIntDemo (int capacity) {
        stack = new int[capacity];
    }
    void push (int value) {
        if (top == stack.length - 1) {
            System.out.println ("Stack overflow");
            return;
        }
        stack[++top] = value;
        System.out.println (value + " pushed into
        stack successfully");
    }
    boolean isEmpty () {
        return top == (-1);
    }
    int pop () {
        if (isEmpty ()) {
            System.out.println ("Stack underflow");
            return -1;
        }
    }
```

```
// return element and update top:-
return stack[top--];
}
```

```
// return top element without removing it:-
int peek () {
```

```
    if (isEmpty ())
        return -1;
    return stack[top];
}
```

```
boolean isFull () {
    return top == stack.length - 1;
}
```

```
void display () {
    if (isEmpty ()) {
        System.out.println ("Stack is empty");
        return;
    }
    System.out.println ("Stack elements (top to
    bottom):");
    for (int i = top; i >= 0; i--)
        System.out.println (stack[i] + " ");
    System.out.println ();
}
```

```
}
public static void main (String[] args) {
    Scanner sc = new Scanner (System.in);
    System.out.println ("Enter stack capacity: ");
    int capacity = sc.nextInt();
```

```

classmate
Date _____
Page _____

if (capacity <= 0) {
    System.out.println("Capacity must be greater
    than zero");
    sc.close();
    return;
}
StackArrayIntDemo stack = new StackArrayIntDemo
    (capacity);

while (true) {
    System.out.println();
    System.out.println("Integer stack using
    array");
    System.out.println("1. Push");
    System.out.println("2. Pop");
    System.out.println("3. Peek");
    System.out.println("4. Display");
    System.out.println("5. Exit");
    System.out.println("Enter your choice");
    int choice = sc.nextInt();
    switch (choice) {
        case 1:
            System.out.println("Enter value to push: ");
            stack.push(sc.nextInt());
            break;
        case 2:
            if (!stack.isEmpty())
                System.out.println(stack.pop() + " popped
                from stack");
    }
}

```

```

classmate
Date _____
Page _____

else
    stack.pop();
break;
case 3:
    if (stack.isEmpty())
        System.out.println("Stack is empty");
    else
        System.out.println("Top element: " +
        stack.peek());
    break;
case 4:
    stack.display();
    break;
case 5:
    System.out.println("Exiting...");
    sc.close();
    return;
default:
    System.out.println("Invalid choice");
}
}
}

```

Generic stack:-

```

→ import java.util.Scanner;
// generic stack for all datatypes:-
public class StackDemoGeneric <T> {
    int max;
    Object[] stack;
    int top = (-1);
}

```

// creating stack of fixed size:-

```
StackDemo.Generic() {
```

```
    this(100);
```

```
}
```

// stack of variable size:-

```
StackDemo.Generic(int capacity) {
```

```
    max = capacity;
```

```
    stack = new Object[max];
```

```
}
```

```
void push (T value) {
```

```
    if (top == max - 1)
```

```
        System.out.println("Stack Overflow");
```

```
    else
```

```
        stack[++top] = value;
```

```
}
```

```
@SuppressWarnings("unchecked")
```

```
T pop() {
```

```
    if (top == (-1))
```

```
        return null;
```

```
    return (T) stack[top--];
```

```
}
```

```
boolean isEmpty() {
```

```
    return top == (-1);
```

```
}
```

// checking if brackets are balanced:-

```
static boolean isMatchingPair (char open, char close) {
```

```
    return (open == '(' && close == ')') || (open == '['
```

```
    && close == ']' ) || (open == '{' && close == '}');
```

```
}
```

```
static boolean isBalanced (String expression) {
```

```
    StackDemo.Generic<Character> stackDemo = new
```

```
    StackDemo.Generic<>(expression.length());
```

```
    for (int i = 0; i < expression.length(); i++) {
```

```
        char ch = expression.charAt(i);
```

```
        if (ch == '(' || ch == '[' || ch == '{')
```

```
            stackDemo.push(ch);
```

```
        else if (ch == ')' || ch == ']' || ch == '}')
```

```
            if (stackDemo.isEmpty())
```

```
                return false;
```

```
            Character open = stackDemo.pop();
```

```
            if (open == null)
```

```
                return false;
```

```
            if (!isMatchingPair(open, ch))
```

```
                return false;
```

```
}
```

```
return stackDemo.isEmpty();
```

```
}
```

```
public static void main (String[] args) {
```

```
    Scanner sc = new Scanner (System.in);
```

```
    // Generic usage examples
```

```
    StackDemo.Generic<Integer> numberStack = new
```

```
    StackDemo.Generic<>();
```

```
    numberStack.push(10);
```

```
    numberStack.push(20);
```

```
    System.out.println ("Popped Integer: " +
```

```
    numberStack.pop());
```

```
    StackDemo.Generic<String> textStack = new
```

```
    StackDemo.Generic<>();
```

textStack.push("hello");
 textStack.push("world");
 System.out.println("Popped string: " + textStack.pop());
 System.out.println("Enter an expression:");
 String expression = sc.nextLine();
 // checks if the expression is balanced:-
 System.out.println(expression + " → " + (isBalanced(expression) ? "Balanced" : "Not Balanced"));
 sc.close();
 }

Queue:-
 → import java.util.Scanner;
 public class QueueDemo {
 int max = 5;
 int[] queue = new int[max];
 int front = (-1);
 int rear = (-1);
 void enqueue(int value) {
 if (rear == max - 1) {
 System.out.println("Queue overflow");
 return;
 }
 if (front == (-1))
 front = 0;
 queue[++rear] = value;
 System.out.println(value + " inserted into queue");
 }
 }

int dequeue() {
 if (isEmpty()) {
 System.out.println("Queue underflow");
 return -1;
 }
 int removed = queue[front++];
 if (front > rear) {
 front = (-1);
 rear = (-1);
 }
 return removed;
 }
 boolean isEmpty() {
 return front == (-1);
 }
 void display() {
 if (isEmpty()) {
 System.out.println("Queue is empty");
 return;
 }
 System.out.println("Queue elements");
 for (int i = front; i <= rear; i++)
 System.out.println(queue[i] + " ");
 System.out.println();
 }
 public static void main (String[] args) {
 Scanner sc = new Scanner(System.in);
 QueueDemo queueDemo = new QueueDemo();
 }

```

while (true) {
    System.out.println("\nQueue Operations: ");
    System.out.println("1. Enqueue");
    System.out.println("2. Dequeue");
    System.out.println("3. Display");
    System.out.println("4. Exit");
    System.out.println("Enter your choice: ");
    int choice = sc.nextInt();
    switch (choice) {
        case 1:
            System.out.println("Enter value to enqueue: ");
            int value = sc.nextInt();
            queueDemo.enqueue(value);
            break;
        case 2:
            int removed = queueDemo.dequeue();
            if (removed != (-1))
                System.out.println(removed + " removed from queue");
            break;
        case 3:
            queueDemo.display();
            break;
        case 4:
            System.out.println("Exiting...");
            sc.close();
            return;
        default:
            System.out.println("Invalid choice");
    }
}

```

Circular Queue:-

```

import java.util.Scanner;
public class CircularQueueDemo {
    int max = 5;
    int[] queue = new int[max];
    int front = (-1);
    int rear = (-1);
    boolean isFull() {
        return (front == 0 && rear == max-1) || (rear+1 == front);
    }
    boolean isEmpty() {
        return front == (-1);
    }
    void enqueue(int value) {
        if (isFull()) {
            System.out.println("Circular Queue Overflow");
            return;
        }
        if (isEmpty()) {
            front = 0;
            rear = 0;
        }
        else if (rear == max-1)
            rear = 0;
        else
            rear++;
        queue[rear] = value;
    }
}

```

```

    System.out.println (value + "inserted into circular
    queue");
}
int dequeue() {
    if (isEmpty()) {
        System.out.println ("Circular queue
        Underflow");
        return -1;
    }
    int removed = queue[front];
    if (front == rear) {
        front = (-1);
        rear = (-1);
    }
    else if (front == max-1)
        front = 0;
    else
        front++;
    return removed;
}
void display() {
    if (isEmpty()) {
        System.out.println ("Circular queue is
        empty");
        return;
    }
    System.out.println ("Circular queue
    elements: ");

```

```

// display elements starting from front:-
int i = front;
while (true) {
    System.out.println (queue[i] + " ");
    if (i == rear)
        break;
    i = (i+1) % max;
}
System.out.println();
}
public static void main (String[] args) {
    Scanner sc = new Scanner (System.in);
    CircularQueueDemo circularQueue = new
    CircularQueueDemo();
    while (true) {
        System.out.println ("In Circular Queue
        Operations: ");
        System.out.println ("1. Enqueue");
        System.out.println ("2. Dequeue");
        System.out.println ("3. Display");
        System.out.println ("4. Exit");
        System.out.println ("5. Enter your
        choice");
        int choice = sc.nextInt();
        switch (choice) {
            case 1:
                System.out.println ("Enter value to
                enqueue");
                int value = sc.nextInt();

```

```
circularQueue.enqueue (value);  
break;  
case 2:  
    int removed = circularQueue.dequeue();  
    if (removed != (-1))  
        System.out.println (removed + " removed  
from circular queue");  
    break;  
case 3:  
    circularQueue.display();  
    break;  
case 4:  
    System.out.println ("Exiting...");  
    sc.close();  
    return;  
default:  
    System.out.println ("Invalid choice");  
} } }
```